20204885 db1 과제

Description 1: 교재의 대학DB 스키마를 분석하고 다음 네 가지 기능을 수용할 수 있도록 스키마 수정 방안을 제시합니다. 각 기능별로 수정 된 스키마와 설명을 함께 기재하고, 수정이 필요하지 않다고 판단할 경우 그 이유를 기재합니다.

해당 문제들은 테이블을 새로 만들거나, 수정하여(수정 또한 create를 하였다) 스키마 수정을 진행하였다.

또한 2주 차의 마지막 부분의 형식을 따랐으며, 값 또한 그 값을 기준으로 진행하였다.

1. 모든 학과(Department)에는 졸업 필수과목들이 존재합니다.

```
create table required(
    dept_name varchar(20) not null,
    course_id varchar(8) not null,
    primary key(course_id, dept_name),
    foreign key(dept_name) references department (dept_name) on delete cascade,
    foreign key(course_id) references course(course_id) on delete cascade
    );
```

required 테이블을 생성하였다. 졸업 필수과목이기 때문에 학과명(dept_name)과 수업 id(course_id)가 반드시 값이 있어야 한다고 판단하였다.

Primary key(이후 PK)의 경우에는, 졸업 필수과목이 학과마다 여러 개일 수 있으므로 dept_name만 사용할 수 없다고 판단하였고, course_id만 따로 사용하지 않은 이유는, 학교에도 있다시피 코드 공유(e.g. 전전 코드 쉐어 등)가 있다고 판단하였기 때문이다. 따라서 둘을 합하여 PK로 활용하였다.

Foreign key(이후 FK)의 경우에는 기존 스키마에 있던 department table에서 dept_name, course table에서 course_id를 가져왔고 둘 다 cascade를 통해서 기존 데이터가 수정되면 연쇄적으로 삭제되도록 하였다.

2. 지도교수(Advisor)와 학과의 정보는 변경될 수 있으나 그 경우 변경 기록이 반드시 DB에 저장되어야 합니다.

```
create table advisor_log(
    s_id varchar(5) not null,
    i_id varchar(5),
    advisor_history numeric(4, 0) not null,
    primary key(s_id, advisor_history),
    foreign key(s_id) references student(ID) on delete cascade,
    foreign key(i_id) references instructor(ID) on delete set null
    );

# 신입생이라 담당교수 없을때를 대비, 학교를 1 만일 이상다닐사람 없으니까 처리완료

# 학과만일때
create table department_log(
    dept_name varchar(20) not null,
    building varchar(15),
    budget numeric(12, 2) check (budget > 0),
    dept_history numeric(4) not null,
    primary key(dept_name, dept_history)
);


# 학생의 학과일때
create table s_department_log(
    s_id varchar(5) not null,
    dept_name varchar(20),
    dept_name_history numeric(4) not null,
    primary key(s_id, dept_name_history),
    foreign key(s_id) references student(ID) on delete cascade
);
# builidng 은 null 허용(일시적으로 건물이 배정되지 않을 수 있음, 온라인만 열리는 학과라던가)
# id 가 varchar 인 이유? varchar 로 id = 0 으로 시작할 수 있기 때문인듯하다.
```

두 번째 문제는 모호한 면이 있어서 총 2가지의 풀이로 진행하였다. 테이블을 기존 테이블을 수정한 게 아니라 log를 따로 만든 이유는, 변경 기록이 저장되어야 한다고 문제에 표기되어 있기 때문이다. 만약 테이블 내에서 바꾼다면 그 변경 기록이 저장되지 않기 때문에 따로 만드는 것으로 판단하였다.

먼저 공통으로 advisor_log라는 table을 새롭게 추가하여 기록이 반드시 저장되어야 한다는 조건을 해결하였다. I_id의 경우 s_id와 다르게 not null 처리를 하지 않았는데, 이는 신입생이라 아직 담당 교수가 없을 때를 대비하였다. 새롭게 advisor_history를 만들었고, numeric(4, 0)으로 구성시켰는데 이는, 인간적으로 담당 교수가 매일 바뀌어도, 학교를 10,000일 이상(약 27.4년) 다닐 수 없다고 판단하였기 때문이다. 3으로 하지 않은 이유는 3년보다는 많이 다닌다고 판단하였기 때문이다.

PK의 경우에는 s_id, advisor_history로 설정하였는데 s_id가 여러 번 중복될 수 있어서 단일 사용 불가(교수님이 바뀌는 경우), 반대로 advisor_history가 아닌 이유는 학생 여러

명이 동일한 값을 가질 수 있기 때문이다.

FK의 경우 기존의 student table에서의 ID, instructor table에서의 ID를 가져왔으며, s_id는 1번 문제와 동일하게 cascade이다. 다만 i_id의 경우 set null을 진행하였는데 이는 중간에 담당 교수가 없는 경우를 표현하기 위함이다(변경 중이거나, 문제 발생 시)

학과의 정보가 변경될 수 있다고 하였는데, 해석이 모호하여 두 가지 테이블을 만들었다. 먼저 학생의 학과일 때인데 이는 위의 advisor_log와 구조가 동일하여 설명을 생략하도록 하겠다. FK를 는 s_id만을 사용하여 나머지는 기록으로 남길 수 있도록 하였다.

다음으로 학과 그 자체일 때인데, 기존 스키마의 department 테이블에 dept_history를 추가한 형식이다. 다만 building의 경우 null을 허용하였는데 이는 온라인 학위제도나, 일시적으로 과의 건물이 할당되지 않는 경우(건물이 파괴되었거나 그런 이유)를 처리하기 위함이다. FK를 사용하지 않은 이유는 위와 동일한 이유이다.

3. 코스(Course)마다 한 번 이상의 시험이 있으며 각 시험은 다수의 질문으로 구성됩니다. 또한 학생들은 각 질문에 대해 답안을 제출합니다.

```sql
create table exam(
    exam_id varchar(8) not null,
    course_id varchar(8) not null,
    exam_type varchar(20),
    number numeric(3) not null check (number > 0),
    primary key(exam_id),
    foreign key(course_id) references course(course_id) on delete cascade
    );
# 따로따로 필요
create table question(
    exam_id varchar(8) not null,
    q_id varchar(8) not null,
    quest varchar(256) not null,
    primary key(exam_id, q_id),
    foreign key(exam_id) references exam(exam_id) on delete cascade
    );

create table answer(
    exam_id varchar(8) not null,
    q_id varchar(8) not null,
    s_id varchar(5) not null,
    answer varchar(1024),
    primary key(exam_id, q_id, s_id),
    foreign key(exam_id, q_id) references question(exam_id, q_id) on delete cascade,
    foreign key(s_id) references student(ID) on delete cascade
    );
```

구조는 수업자료를 참고하여 만들었다. 전체적인 구조는 exam table에서 시험 그 자체를 관리하며 문제와 답을 따로 table을 구성하였다. 이는 문제에 대해 학생들의 답을 따로 관리하기 위함이다.

먼저 exam table의 경우 exam_id를 PK로 하여 시험을 유일하게 구분하였으며 코스마다 한 번 이상의 시험이 있다고 하였으므로 check를 사용하여 number를 확인하였다. FK는 course id를 가져왔으며 이는 course table의 course_id이다.

question table의 경우 q_id는 문제번호, quest의 경우 문제 내용이다. 정답은 따로 저장하지 않았는데 이는 답이 바뀌는 경우가 있기 때문에 시험을 낸 주체가 따로 관리해야 한다고 판단하였기 때문이다. PK의 경우 exam_id마다 여러 개의 문제가 있고, 문제는 1번부터인 게 시험마다인 게 많기 때문에 두 개를 조합하여 PK를 만들었다. exam_id의 경우 새롭게 만든 exam table에서 가져왔다.

answer table의 경우, question과 비슷한 구조이며, q_id는 문제번호, s_id는 학번, answer는 적은 답이다. 다만 여기서 not null을 지정하지 않은 이유는 백지로 낼 수 있기 때문이다. PK의 경우 3개로 합쳐서 구성하였는데 이는 같은 시험에 같은 문제에 학생들이 답을 하기 때문이다. 반대로 한 학생은 같은 시험에 여러 대답을 하기 때문에 단독으로 사

용할 수 없으며 문제번호도 그러한 이유이다.

FK는 question table에서 exam_id, q_id를 가져왔으며 기존의 student table의 ID를 s_id로 가져왔다. 둘 다 제거되면 연쇄적으로 작용시키기 위해 cascade를 적용하였다.

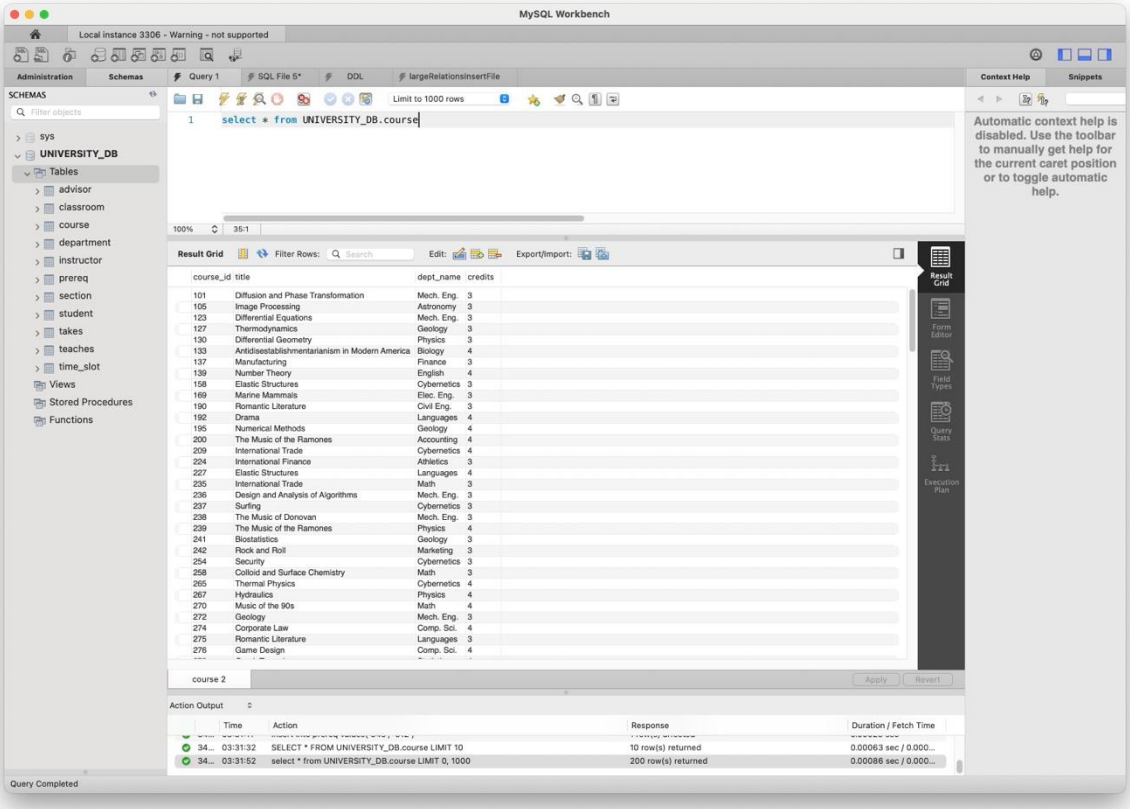
4. 외국인 학생은 어떤 코스든 수강할 수 있습니다. 그러나 외국인 학생은 지도교수가 배정되지 않습니다.

```
create table foreigner(
    f_id varchar(5) not null,
    primary key(f_id),
    foreign key(f_id) references student(id) on delete cascade
);
create table domestic(
    d_id varchar(5) not null,
    primary key(d_id),
    foreign key(d_id) references student(id) on delete cascade
);
create table advisor(
    d_id varchar(5) not null,
    i_id varchar(5),
    primary key(d_id),
    foreign key(d_id) references domestic(d_id) on delete cascade,
    foreign key(i_id) references instructor(id) on delete cascade
);
create table takes(
    id varchar(5) not null,
    course_id varchar(8) not null,
    sec_id varchar(8) not null,
    semester varchar(6) not null,
    year numeric(4,0) not null,
    grade varchar(2),
    primary key (id, course_id, sec_id, semester, year),
    foreign key (id) references student(id) on delete cascade,
    foreign key (course_id, sec_id, semester, year) references section(course_id, sec_id,
semester, year) on delete cascade
);
```

처음에는 student table에 한 번에 관리하고 싶었는데, 처리할 방법이 생각나지 않아 따로따로 관리하게 수정하였다. student table에서 domestic과 foreigner로 구분하여 각각 국내학생과, 외국인 학생으로 구분하여 처리할 수 있도록 하였다. 둘 다 속성명들을 변경하여 좀 더 이해하기 위해 편하게 수정하였다. PK의 경우 둘 다 학생 id이다.

advisor table의 경우에는 수정을 진행하였는데, 다른 건 기존의 테이블과 비슷하나 큰 변경점은 FK로 student를 받는 게 아닌 student를 받은 domestic의 d_id를 받는 것이다. 이를 통해서 외국인 학생은 지도교수가 배정되지 않는 점을 구현하였다.

다음으로 외국인 학생의 코스 수강에 대해서 처리하기 위해 takes를 처리하고 싶었는데,

일단 DB에서 수강에 학과 제한이 있다고 하지 않았으므로(경험적으로는 국내 학생들은 제한이 있는 것이 맞다. 문제의 관념적 의미에서도 그렇다), 외국인 학생은 어떤 코스든 수강할 수 있다고 하는 조건을 테이블을 수정하지 않고 처리하였다. 현재까지 배운 내용으로는 처리가 불가능할 것 같고, 다음에 조건을 거는법을 배운다면(check는 같은 tuple 내에서만 처리 가능하다고 확인하였다) 이후에 처리할 수 있을 것이다.

Description 2: MySQL 개발환경을 구축하고 DB-book 웹사이트에서 제공하는 스크립트를 통해 대학DB를 생성하여 그 결과를 첨부합니다.